Inventory & Order Management System

Mid-Semester Evaluation Report

Project overview

Small businesses can lose track of stock and customer orders when information is handled manually. This project provides a focused web application that stores products, displays stock levels, flags low-stock products, confirms customer orders only when inventory is available, and retains an audit trail of quantity changes.

Scope

In scope: product creation and maintenance, inventory visibility, low-stock status, restocking, multi-line order confirmation, stock validation, order history, and search. **Out of scope:** login, payments, suppliers, multiple warehouses, barcode scanning, and external integrations.

Stakeholders

Stakeholder	Primary need
Inventory manager	Accurate stock visibility and low-stock warnings.
Order-processing staff	A quick way to confirm orders without overselling.
Business owner/administrator	Reliable records of products, orders, and stock movements.
Course evaluator	Traceable requirements, a working feature, tests, and design evidence.

Functional requirements

ID	Requirement	Priority
FR-01	Create products with unique SKU, price, opening stock, and reorder level.	Must
FR-02	View active inventory and identify low-stock products.	Must
FR-03	Restock products while recording a stock movement.	Must
FR-04	Create multi-line orders and validate combined requested stock.	Must
FR-05	Confirm an order, deduct stock, and display the completed order.	Must
FR-06	Search inventory and order history; edit or deactivate products.	Should

Architecture and database design

The system uses Node.js with Express for the web interface, EJS for server-rendered pages, and SQLite for persistent local storage. The order service owns transactional confirmation; the inventory service owns product creation, restocking, editing, and deactivation.

Table	Purpose and key fields
products	SKU, name, unit price, stock quantity, reorder level, active flag.
orders	Customer name, status, total amount, creation timestamp.
order_items	Order, product, quantity, and captured unit price.
stock_movements	Product, optional order, signed quantity change, reason, timestamp.

Integrity decisions

Foreign keys are enabled. Product SKUs are unique. Prices and stock values cannot be negative. Duplicate lines in an order are combined before availability is checked. The order header, order items, inventory deductions, and negative stock movements are committed in one database transaction, so a rejected order leaves no partial data.

Use cases

Confirm customer order: staff enter a customer name, select one or more products and quantities, then confirm. The system validates stock, creates the order and items, deducts stock, creates movement records, and displays a confirmation. If any line exceeds stock, it displays an error and stores nothing.

Restock product: an inventory manager enters a positive quantity next to a product. The system increases stock and stores a positive stock movement.

Non-functional requirements

The project prioritises transactional stock integrity, clear validation feedback, a small number of understandable screens, modular code, local portability, and automated tests.

Verification and delivery status

The current automated suite contains **eight passing tests**. It verifies product creation, restocking, editing and deactivation, successful order confirmation, insufficient-stock rejection, duplicate-line validation, and two HTTP-level end-to-end order flows. Run **npm test** against the final source snapshot immediately before submission and attach the actual output or a screenshot.

Demonstration plan

Time	Demonstration step
0:00-1:15	Introduce dashboard, inventory fields, and low-stock state.
1:15-2:25	Add or restock a product and explain auditability.
2:25-4:10	Create a multi-line order, open confirmation, and show reduced stock.
4:10-5:15	Attempt an unavailable quantity, then show order history and search.
5:15-6:30	Explain the transaction design and show real test output.

Project planning and contributions

Team names, allocation, meeting records, and contribution evidence are intentionally not included because they must be completed from real team activity. The repository contains a contribution-record template for the team to fill in before submission.

Submission checklist

[ ] Review requirements with the team. [ ] Fill the contribution record from actual evidence. [ ] Re-run and capture test output. [ ] Add final wireframes or approved screenshots. [ ] Record a 5-7 minute demonstration. [ ] Export and submit this report PDF with the source-code snapshot.